

Excercises Lab 4 Bonus

Hash Functions - MACs

1 Blake Collision

1.1 Problem

The objective is to find a collision for a specific hash function, denoted as $H(x)$. The function is defined based on the standard MD5 algorithm but with a significant modification: the output is truncated to the first 2 bytes.

The definition of the function is:

$$H(x) = \text{MD5}(x)[0 : 2] \quad (1)$$

where $\text{MD5}(x)$ denotes the 128-bit (16-byte) output of the MD5 hash of input x .

1.2 Search Space Analysis

Standard MD5 produces a 128-bit hash (2^{128} possibilities), making brute-force collision attacks computationally infeasible. However, $H(x)$ retains only 16 bits.

The size of the output space for $H(x)$ is:

$$N = 2^{16} = 65,536$$

This relatively small output space implies that collisions are highly probable after a small number of trials.

1.3 Solution Approach

To find a collision efficiently, we utilize the **Birthday Paradox**. While the pigeonhole principle guarantees a collision after $N+1$ (65,537) attempts, the probability of a collision reaches 50% much earlier, approximately at $\sqrt{N}$ attempts.

$$\text{Expected attempts} \approx \sqrt{65,536} = 256$$

1.4 Algorithm

I implemented a Python script to perform the collision search. The algorithm operates as follows:

1. Initialize an empty dictionary (hash map) called `seen_hashes` to store pairs of $\langle \text{hash}, \text{input} \rangle$.

2. Initialize a counter $i = 0$.
3. Enter a loop:
 - Convert i to a string to serve as the input x .
 - Compute the truncated hash $h = H(x)$.
 - Check if h exists as a key in `seen_hashes`.
 - **If found:** The program has found two inputs, x_{old} (from the dictionary) and x_{new} (current i), such that $H(x_{old}) = H(x_{new})$. Report the collision and terminate.
 - **If not found:** Store h in the dictionary with value x , increment i , and continue.

```
1 import hashlib
2
3 def H(x):
4     if isinstance(x, str):
5         x = x.encode('utf-8')
6     md5 = hashlib.md5(x).digest()
7     return md5[:2]
8
9
10 def find_collision():
11     print("Searching for a collision...")
12     seen_hashes = {}
13     counter = 0
14     while True:
15         current_input = str(counter)
16         hash_val = H(current_input)
17         if hash_val in seen_hashes:
18             previous_input = seen_hashes[hash_val]
19             # Collision found!
20             print(f"\n--- Collision Found! ---")
21             print(f"Input 1: '{previous_input}'")
22             print(f"Input 2: '{current_input}'")
23             print(f"Truncated Hash (Hex): {hash_val.hex()}")
24
25             print(f"Attempts needed: {counter}")
26             return
27
28         seen_hashes[hash_val] = current_input
29         counter += 1
30
31 if __name__ == "__main__":
32     find_collision()
```

This method is efficient because dictionary lookups are $O(1)$ on average. The script finds a collision almost instantly due to the small collision bound of the truncated hash.